Nested Varying Effects

General Principles

To model hierarchies where groups are themselves nested within larger units (e.g., students within classes, which are within schools), we use *Nested Varying Effects*. This allows the model to share information across different levels of the hierarchy, improving estimates through multilevel pooling.

Note

> **Note**
>
> - We have the same considerations as for Varying Intercepts and Varying Slopes.
> - In a nested model, a parameter at one level (e.g., a group-level intercept) becomes the mean for the distribution of parameters at a lower level (e.g., individual-level intercepts).
> - This structure is often represented using multiple indices: j for the group and $k(j)$ for the super-group (e.g., region) that group j belongs to.
> - To capture the correlation between multiple varying effects (e.g., intercept and slope) at any level of the hierarchy, we use a **Multivariate Normal (MVN)** distribution.
> - Non-centered parameterization is highly recommended for nested models to avoid "funnel" geometries that can hinder MCMC sampling.

Example

Below is an example of a nested model with both **varying intercepts** and **varying slopes**. We model the outcome y for individuals in `groups` nested within `regions`. The relationship between x and y varies at both levels.

Python (Centered)

```python
from BayesForge import bf
import jax.numpy as jnp
import numpy as np

# Setup device ------------------------------------------------
m = bf(platform='cpu')

# Load data - column args are mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=True)
m.data(data_path)

# Define model ------------------------------------------------
def model_nested(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region level
    mu_global = jnp.stack([m.dist.normal(5, 2,  name='global_intercept'),
                           m.dist.normal(-1, 1, name='global_beta')])
    sigma_reg  = m.dist.exponential(1, shape=(2,), name='sigma_region')
    corr_reg   = m.dist.lkj(2, 2, name='corr_region')
    cov_reg    = jnp.diag(sigma_reg) @ corr_reg @ jnp.diag(sigma_reg)
    region_effects = m.dist.multivariate_normal(
        mu_global, cov_reg, shape=(N_regions,), name='region_effects'
    )

    # 2. Group level - parent mapping via JAX scatter (traceable under pmap)
    group_to_region = jnp.zeros(N_groups, dtype=jnp.int32).at[group_id].set(region_id)
    sigma_grp  = m.dist.exponential(1, shape=(2,), name='sigma_group')
    corr_grp   = m.dist.lkj(2, 2, name='corr_group')
    cov_grp    = jnp.diag(sigma_grp) @ corr_grp @ jnp.diag(sigma_grp)
    group_effects = m.dist.multivariate_normal(
        region_effects[group_to_region], cov_grp, name='group_effects'
    )

    mu_est = group_effects[group_id, 0] + group_effects[group_id, 1] * x
    m.dist.normal(mu_est, sigma, obs=y)

# Run sampler ------------------------------------------------
m.fit(model_nested, num_samples=1000, num_warmup=500, num_chains=1)
```

```
m.summary()
```

```
/home/sosa/.local/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning:

IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs

An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 32

/home/sosa/.local/lib/python3.13/site-packages/jax/_src/ops/scatter.py:105: FutureWarning:

scatter inputs have incompatible types: cannot safely cast value from dtype=int64 to dtype=in

  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   5%|          | 75/1500 [00:03<01:12, 19.70i
/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide

/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
corr_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
corr_group[0, 1]	0.43	0.23	0.04	0.77	639.39	600.14	NaN	0.01	0.01
corr_group[1, 0]	0.43	0.23	0.04	0.77	639.39	600.14	NaN	0.01	0.01
corr_group[1, 1]	1.00	0.00	1.00	1.00	1104.53	1014.71	NaN	0.00	0.00
corr_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...
sigma	0.49	0.02	0.46	0.52	1657.63	729.35	NaN	0.00	0.00
sigma_group[0]	0.42	0.09	0.30	0.59	673.81	610.61	NaN	0.00	0.00
sigma_group[1]	0.25	0.06	0.17	0.37	514.32	577.13	NaN	0.00	0.00
sigma_region[0]	1.03	0.43	0.57	1.80	765.21	715.54	NaN	0.02	0.03
sigma_region[1]	0.46	0.23	0.22	0.88	673.73	652.34	NaN	0.01	0.01

Python (Non-Centered)

```
# Data and m object carry over from the Centered tab above.

def model_nested_noncentered(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region level
    mu_global    = jnp.stack([m.dist.normal(5, 2,  name='global_intercept'),
                              m.dist.normal(-1, 1, name='global_beta')])
    sigma_region = m.dist.exponential(1, shape=(2,), name='sigma_region')
    L_region     = m.dist.lkj_cholesky(2, 2, name='L_region')
    z_region     = m.dist.normal(0, 1, name='z_region', shape=(2, N_regions))
    region_effects = mu_global + ((sigma_region[..., None] * L_region) @ z_region).T

    # 2. Group level - parent mapping via JAX scatter (traceable under pmap)
    pid          = jnp.zeros(N_groups, dtype=jnp.int32).at[group_id].set(region_id)
    sigma_group  = m.dist.exponential(1, shape=(2,), name='sigma_group')
    L_group      = m.dist.lkj_cholesky(2, 2, name='L_group')
    z_group      = m.dist.normal(0, 1, name='z_group', shape=(2, N_groups))
    group_effects = region_effects[pid] + ((sigma_group[..., None] * L_group) @ z_group).T

    mu_est = group_effects[group_id, 0] + group_effects[group_id, 1] * x
    m.dist.normal(mu_est, sigma, obs=y)

m.fit(model_nested_noncentered, num_samples=1000, num_warmup=500, num_chains=1)
m.summary()
```

```
/home/sosa/.local/lib/python3.13/site-packages/jax/_src/ops/scatter.py:105: FutureWarning:

scatter inputs have incompatible types: cannot safely cast value from dtype=int64 to dtype=i

  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   5%|          | 75/1500 [00:02<00:50, 28.33i
/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide

/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
L_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
L_group[0, 1]	0.00	0.00	0.00	0.00	1000.00	1000.00	NaN	0.00	NaN
L_group[1, 0]	0.41	0.24	-0.02	0.74	545.88	652.57	NaN	0.01	0.01
L_group[1, 1]	0.87	0.11	0.67	1.00	560.60	652.57	NaN	0.00	0.00
L_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...
z_region[1, 0]	-0.69	0.67	-1.77	0.35	763.53	587.22	NaN	0.02	0.02
z_region[1, 1]	1.03	0.69	-0.07	2.18	876.26	638.43	NaN	0.02	0.02
z_region[1, 2]	-0.61	0.65	-1.63	0.43	737.90	620.41	NaN	0.02	0.02
z_region[1, 3]	0.37	0.79	-0.93	1.60	1013.82	808.20	NaN	0.02	0.03
z_region[1, 4]	-0.04	0.67	-1.09	1.09	733.51	710.61	NaN	0.02	0.02

Python (Built-in)

```python
from BayesForge import bf
import jax.numpy as jnp

# Setup device -------------------------------------------------
m = bf(platform='cpu')

# Load data - column args are mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=True)
m.data(data_path)

# group_ids: one obs-level index array per level, top to bottom.
# Parent structure is derived automatically from these arrays.
def model_nested_builtin(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    a_g_est, b_g_est = m.effects.nested_varying_effects(
        N_vars=2,
        names=["region", "group"],
        N_groups=[N_regions, N_groups],
        group_ids=[region_id, group_id],
        centered=False,
    )

    mu_est = a_g_est + b_g_est * x
```

```
    m.dist.normal(mu_est, sigma, obs=y)

m.fit(model_nested_builtin, num_samples=1000, num_warmup=500, num_chains=1)
m.summary()
```

```
/home/sosa/.local/lib/python3.13/site-packages/jax/_src/ops/scatter.py:105: FutureWarning:

scatter inputs have incompatible types: cannot safely cast value from dtype=int64 to dtype=i

jax.local_device_count 32

  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   5%|          | 75/1500 [00:02<00:44, 31.77i
/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide

/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_s
L_corr_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
L_corr_group[0, 1]	0.00	0.00	0.00	0.00	1000.00	1000.00	NaN	0.00	NaN
L_corr_group[1, 0]	0.41	0.23	0.02	0.75	730.70	686.42	NaN	0.01	0.01
L_corr_group[1, 1]	0.87	0.11	0.67	1.00	741.71	754.34	NaN	0.00	0.00
L_corr_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...
z_region[1, 0]	-0.76	0.63	-1.78	0.19	777.85	772.98	NaN	0.02	0.02
z_region[1, 1]	0.95	0.68	-0.04	2.09	618.75	691.42	NaN	0.03	0.02
z_region[1, 2]	-0.65	0.68	-1.74	0.41	619.33	608.18	NaN	0.03	0.02
z_region[1, 3]	0.31	0.81	-1.04	1.63	838.17	637.53	NaN	0.03	0.03
z_region[1, 4]	-0.09	0.68	-1.17	1.03	631.84	684.18	NaN	0.03	0.02

Python (Built-in Centered)

```
from BayesForge import bf
import jax.numpy as jnp
```

```python
# Setup device ------------------------------------------------
m = bf(platform='cpu')

# Load data - column args are mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=True)
m.data(data_path)

def model_nested_builtin_centered(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    a_g_est, b_g_est = m.effects.nested_varying_effects(
        N_vars=2,
        names=["region", "group"],
        N_groups=[N_regions, N_groups],
        group_ids=[region_id, group_id],
        centered=True,
    )

    mu_est = a_g_est + b_g_est * x
    m.dist.normal(mu_est, sigma, obs=y)

m.fit(model_nested_builtin_centered, num_samples=1000, num_warmup=500, num_chains=1)
m.summary()
```

```
jax.local_device_count 32

/home/sosa/.local/lib/python3.13/site-packages/jax/_src/ops/scatter.py:105: FutureWarning:

scatter inputs have incompatible types: cannot safely cast value from dtype=int64 to dtype=i

  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   5%|          | 75/1500 [00:02<00:43, 32.85i
/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide

/home/sosa/.local/lib/python3.13/site-packages/arviz_stats/base/diagnostics.py:313: RuntimeWa

invalid value encountered in scalar divide
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
corr_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
corr_group[0, 1]	0.43	0.23	0.04	0.77	761.44	636.91	NaN	0.01	0.01
corr_group[1, 0]	0.43	0.23	0.04	0.77	761.44	636.91	NaN	0.01	0.01
corr_group[1, 1]	1.00	0.00	1.00	1.00	874.90	851.35	NaN	0.00	0.00
corr_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...
sigma	0.49	0.02	0.46	0.52	1427.22	579.29	NaN	0.00	0.00
sigma_group[0]	0.42	0.09	0.30	0.57	662.48	643.23	NaN	0.00	0.00
sigma_group[1]	0.25	0.07	0.17	0.37	524.30	612.52	NaN	0.00	0.00
sigma_region[0]	1.11	0.51	0.58	2.04	585.55	481.40	NaN	0.02	0.03
sigma_region[1]	0.46	0.23	0.21	0.87	616.23	627.73	NaN	0.01	0.01

Julia

```julia
using PyBayesForge

# Setup device ------------------------------------------------
m = importBF(platform="cpu")

# Load data - column args mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=true)
m.data(data_path)

# Define model ------------------------------------------------
@BF function model(y, x, group_id, region_id, N_groups=20, N_regions=5)
    sigma = m.dist.exponential(1, name="sigma")

    a_g_est, b_g_est = m.effects.nested_varying_effects(
        N_vars    = 2,
        names     = ["region", "group"],
        N_groups  = [N_regions, N_groups],
        group_ids = [region_id, group_id],
        centered  = false,
    )

    mu = a_g_est .+ b_g_est .* x
    m.dist.normal(mu, sigma, obs=y)
end
```

```
# Run mcmc -----------------------------------------------
m.fit(model)
m.summary()
```

R

```
library(BayesForge)

# Setup platform -----------------------------------------------
m = importBF(platform='cpu')

# Load data - column args mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m$load$sim_nested_effects(only_path=TRUE)
m$data(data_path)

# Define model -------------------------------------------------
model <- function(y, x, group_id, region_id, N_groups=20L, N_regions=5L) {
  sigma = m$dist$exponential(1, name='sigma')

  veff = m$effects$nested_varying_effects(
    N_vars    = 2L,
    names     = list('region', 'group'),
    N_groups  = list(N_regions, N_groups),
    group_ids = list(region_id, group_id),
    centered  = FALSE,
  )
  a_g_est = veff[[1]]
  b_g_est = veff[[2]]

  mu = a_g_est + b_g_est * x
  m$dist$normal(mu, sigma, obs=y)
}

# Run MCMC -----------------------------------------------------
m$fit(model)
m$summary()
```

Mathematical Details

We model the outcome Y_i for observation i. The intercept α_j and slope β_j for group $j(i)$ are nested within region $k(j)$.

$$Y_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha_{[j(i)]} + \beta_{[j(i)]} X_i$$

Centered Parameterization

The vector of group-level effects follows a Multivariate Normal distribution centered on the region-level effects:

$$\begin{bmatrix} \alpha_j \\ \beta_j \end{bmatrix} \sim \text{MVNormal}\left(\begin{bmatrix} \gamma_{\alpha,[k(j)]} \\ \gamma_{\beta,[k(j)]} \end{bmatrix}, \Sigma_{group} \right)$$

The region-level effects themselves follow their own Multivariate Normal distribution:

$$\begin{bmatrix} \gamma_{\alpha,k} \\ \gamma_{\beta,k} \end{bmatrix} \sim \text{MVNormal}\left(\begin{bmatrix} \bar{\gamma}_\alpha \\ \bar{\gamma}_\beta \end{bmatrix}, \Sigma_{region} \right)$$

Where: - $\Sigma = \mathbf{SRS}$ (diagonal matrix of standard deviations $\mathbf{S}$ and correlation matrix $\mathbf{R}$). - $\bar{\gamma}_\alpha, \bar{\gamma}_\beta$ are the global average intercept and slope. - $\Sigma_{region}, \Sigma_{group}$ are the covariance matrices for each hierarchical level.

Non-Centered Parameterization

To avoid funnel geometries, the non-centered formulation instead uses Cholesky factors L_{region} and L_{group} alongside standardized, non-correlated normal variables:

$$\begin{bmatrix} z_{\alpha,k} \\ z_{\beta,k} \end{bmatrix} \sim \text{Normal}(0,1), \quad \begin{bmatrix} z_{\alpha,j} \\ z_{\beta,j} \end{bmatrix} \sim \text{Normal}(0,1)$$

The region-level and group-level distributions are linearly transformed:

$$\begin{bmatrix} \gamma_{\alpha,k} \\ \gamma_{\beta,k} \end{bmatrix} = \begin{bmatrix} \bar{\gamma}_\alpha \\ \bar{\gamma}_\beta \end{bmatrix} + \text{diag}(\mathbf{S}_{region}) L_{region} \begin{bmatrix} z_{\alpha,k} \\ z_{\beta,k} \end{bmatrix}$$

$$\begin{bmatrix} \alpha_j \\ \beta_j \end{bmatrix} = \begin{bmatrix} \gamma_{\alpha,[k(j)]} \\ \gamma_{\beta,[k(j)]} \end{bmatrix} + \text{diag}(\mathbf{S}_{group}) L_{group} \begin{bmatrix} z_{\alpha,j} \\ z_{\beta,j} \end{bmatrix}$$

Model validation tests

- Parameter recovery: `builtin_function.py` simulates nested data with known parameters, fits the built-in nested model and checks that the parameters are recovered.
- Documentation example: the Python example of this page is run by the documentation tests (`test_model_pages.py`).

Notes

> **Note**
>
> - By nesting the **MVN** prior, we allow the model to learn how the correlation between intercepts and slopes persists across different levels of the hierarchy.
> - The non-centered parameterization (used by default in the `m.effects.nested_varying_effects` built-in function) is critical for convergence in these complex models.
> - This approach can be generalized to N levels and M variables by increasing the dimensionality of the vectors and matrices.

Reference(s)